FRIDAY, JUNE 29, 2012

Builder Design pattern in Java - Example Tutorial

Builder design pattern in Java is a creational pattern i.e. used to create objects, similar to factory method design pattern which is also creational design pattern. Before learning any design pattern I suggest find out the problem a particular design pattern solves. Its been well said **necessity is mother on invention**. learning design pattern without facing problem is not that effective, Instead if you have already faced issues than its much easier to understand design pattern and learn how its solve the issue. In this Java design pattern tutorial we will first see what problem *Builder design pattern* solves which will give some insight on **when to use builder design pattern in Java,** which is also a popular design pattern interview question and then we will see example of Builder design pattern and pros and cons of using Builder pattern in Java.

What problem Builder pattern solves in Java

As I said earlier *Builder pattern* is a creational design pattern it means its solves problem related to object creation. Constructors in Java are used to create object and can take parameters required to create object. Problem starts when an Object can be created with **lot of parameters**, some of them may be **mandatory** and others may be **optional**. Consider a class which is used to create `Cake`, now you need number of item like `egg`, `milk`, `flour` to create cake. many of them are mandatory and some of them are optional like `cherry`, `fruits` etc. If we are going to have overloaded constructor for different kind of cake then there will be many constructor and even worst they will accept many parameter.

Problems:
1) too many constructors to maintain.
2) error prone because many fields has same type e.g. `sugar` and and `butter` are in cups so instead of 2 cup sugar if you pass 2 cup butter, your compiler will not complain but will get a buttery cake with almost no sugar with high cost of wasting butter.

You can partially solve this problem by creating `Cake` and then adding ingredients but that will impose another problem of **leaving Object on inconsistent state during building**, ideally cake should not be available until its created. Both of these problem can be solved by using Builder design pattern in Java. Builder design pattern not only improves readability but also reduces chance of error by adding ingredients explicitly and making object available once fully constructed.

By the way there are many design pattern tutorial already there in Javarevisited like Decorator pattern tutorial and Observer pattern in Java. If you haven't read them already then its worth looking.

Example of Builder Design pattern in Java

We will use same example of creating Cake using Builder design pattern in Java. here we have static nested builder class inside `Cake` which is used to create object.

Guidelines for Builder design pattern in Java
1) Make a static nested class called Builder inside the class whose object will be build by `Builder`. In this example its `Cake`.

2) Builder class will have exactly same set of fields as original class.
3) Builder class will expose method for adding ingredients e.g. `sugar()` in this example. each method will return same `Builder` object. Builder will be enriched with each method call.

4) `Builder.build()` method will copy all builder field values into actual class and return object of `Item` class.
5) Item class (class for which we are creating Builder) should have private constructor to create its object from `build()` method and prevent outsider to access its constructor.

```
public class BuilderPatternExample {

    public static void main(String args[]) {

        //Creating object using Builder pattern in java
        Cake whiteCake = new Cake.Builder().sugar(1).butter(0.5).
eggs(2).vanila(2).flour(1.5). bakingpowder(0.75).milk(0.5).build();

        //Cake is ready to eat :)
        System.out.println(whiteCake);
    }
}

class Cake {

    private final double sugar;   //cup
    private final double butter;  //cup
    private final int eggs;
    private final int vanila;     //spoon
```

```
    private final double flour;   //cup
    private final double bakingpowder; //spoon
    private final double milk;  //cup
    private final int cherry;

    public static class Builder {

        private double sugar;   //cup
        private double butter;  //cup
        private int eggs;
        private int vanila;     //spoon
        private double flour;   //cup
        private double bakingpowder; //spoon
        private double milk;  //cup
        private int cherry;

        //builder methods for setting property
        public Builder sugar(double cup){this.sugar = cup; return this; }
        public Builder butter(double cup){this.butter = cup; return this; }
        public Builder eggs(int number){this.eggs = number; return this; }
        public Builder vanila(int spoon){this.vanila = spoon; return this; }
        public Builder flour(double cup){this.flour = cup; return this; }
        public Builder bakingpowder(double spoon){this.sugar = spoon; return this; }
        public Builder milk(double cup){this.milk = cup; return this; }
        public Builder cherry(int number){this.cherry = number; return this; }


        //return fully build object
        public Cake build() {
            return new Cake(this);
        }
    }

    //private constructor to enforce object creation through builder
    private Cake(Builder builder) {
        this.sugar = builder.sugar;
        this.butter = builder.butter;
        this.eggs = builder.eggs;
        this.vanila = builder.vanila;
        this.flour = builder.flour;
        this.bakingpowder = builder.bakingpowder;
        this.milk = builder.milk;
        this.cherry = builder.cherry;
    }

    @Override
    public String toString() {
        return "Cake{" + "sugar=" + sugar + ", butter=" + butter + ", eggs=" + eggs + ",
vanila=" + vanila + ", flour=" + flour + ", bakingpowder=" + bakingpowder + ", milk=" +
milk + ", cherry=" + cherry + '}';

    }

}
```

Output:

```
Cake{sugar=0.75, butter=0.5, eggs=2, vanila=2, flour=1.5, bakingpowder=0.0, milk=0.5,
cherry=0}
```

Builder design pattern in Java – Pros and Cons

Live everything Builder pattern also has some disadvantages, but if you look at below, advantages clearly outnumber disadvantages of Builder design pattern. Any way here are few advantages and disadvantage of Builder design pattern for creating objects in Java.

Advantages:

1) more maintainable if number of fields required to create object is more than 4 or 5.
2) less error-prone as user will know what they are passing because of explicit method call.
3) more robust as only fully constructed object will be available to client.

Disadvantages:

1) verbose and code duplication as Builder needs to copy all fields from Original or Item class.

When to use Builder Design pattern in Java

Builder Design pattern is a creational pattern and should be used when number of parameter required in constructor is more than manageable usually 4 or at most 5. Don't confuse with **Builder and Factory pattern** there is an obvious difference between Builder and Factory pattern, as Factory can be used to create different implementation of same interface but Builder is tied up with its Container class and only returns object of Outer class.

That's all on **Builder design pattern in Java**. we have seen why we need Builder pattern , what problem it solves, Example of builder design pattern in Java and finally when to use Builder patter with pros and cons. So if you are not using telescoping constructor pattern or have a choice not to use it than Builder pattern is way to go.